

DKIM Bulk Signature Verifier

User Guide & Options Reference

Version 1.7.1 | Build 2026-11-05 | verify_dkim_bulk.py

Bulk DKIM verification of .eml evidence files

Every DKIM signature verified - RSA and Ed25519 (RFC 8463)
Three public-key sourcing modes (live DNS or preserved local key files)
t= / x= / i= tag evaluation and line-ending normalisation
Every public key used preserved, with a key manifest

Author: Jacques Boucher (jjrboucher@gmail.com) | github.com/jjrboucher/DKIM-Validator
Software: Apache License 2.0 | Documentation: CC BY 4.0 | Provided as-is, without warranty

Contents

1. Overview 3

2. Requirements and Installation 4

3. The Graphical Interface 4

4. Public Key Modes in Detail 6

5. Messages with Multiple DKIM Signatures 8

6. Keyword Alerts 10

7. Command-Line Reference 11

8. How Verification Works 12

9. Output and Interpreting Results 13

10. Known Limitations (v1.7.1) 16

11. Version History 16

12. License, Warranty and Repository 17

Documentation licensed under CC BY 4.0

1. Overview

DKIM Bulk Signature Verifier is a Python tool that verifies the DomainKeys Identified Mail (DKIM) signatures of every `.eml` file in a folder. For each signature it recomputes the body hash, rebuilds the signed header block, checks the cryptographic signature against the signing domain's public key and evaluates the signature's identity and time tags. Results are written to two CSV ledgers, supported by per-message log files, optional extracted artifacts and a copy of every public key used.

A valid DKIM signature shows that the signed headers and body have not been altered since the message was signed by the domain in the d= tag. In an investigation this helps establish whether an email produced as evidence is an unaltered copy of what the signing server sent.

What the tool does for each .eml file

- Calculates MD5 and SHA-256 hashes of the original file (evidence integrity).
- Normalises LF-only or mixed line endings to CRLF on a working copy; the original file is never changed.
- Optionally scans the message body against regex keyword patterns and records any matches.
- Finds **every** `DKIM-Signature` header and, for each one:
  - locates the public key according to the selected key mode;
  - canonicalises the body (simple or relaxed), computes the body hash and compares it with bh=;
  - rebuilds the signed header block from the h= list and verifies the b= signature;
  - checks that i= is within d=, and reports the t= signing time and x= expiry.
  - saves a copy of the public key it used to the keys/ folder.
- Reports PASS if at least one signature passes, and writes per-signature details and a log file (see Section 5 for messages with several signatures).

Supported algorithms and key formats

Item	Supported
Signature algorithms	rsa-sha256; rsa-sha1 (deprecated, flagged in Notes); ed25519-sha256 (RFC 8463)
Canonicalisation	simple/simple, relaxed/relaxed and mixed combinations (c= tag); defaults to simple/simple
Body length limit	l= tag honoured; the number of unsigned trailing bytes is reported in Notes
Tags evaluated	i= (identity), t= (signing time), x= (expiry)
Key formats	Raw DNS TXT record (v=DKIM1; k=...; p=...), zone-file style line, bare base64, PEM (BEGIN PUBLIC KEY / BEGIN RSA PUBLIC KEY), DER SubjectPublicKeyInfo, PKCS#1, raw 32-byte Ed25519 key
Revoked keys	An empty p= tag is detected and reported as a revoked key

Documentation licensed under CC BY 4.0

2. Requirements and Installation

Component	Purpose	Notes
Python 3	Runtime	Run with python / python3
cryptography	RSA / Ed25519 verification, key loading	Required. The script exits with an install hint if missing.
tkinter	Graphical interface	Bundled with most Windows/macOS Python installs; on Linux install python3-tk. Not needed with --no-gui.
Internet (HTTPS)	Mode 1 live DNS lookups	Only needed for Mode 1. Modes 2 and 3 work fully offline.

Get the tool and install the dependency:

```
git clone https://github.com/jjrboucher/DKIM-Validator.git
cd DKIM-Validator
pip install cryptography
```

Launching the tool

Launch method	Command	Result
Graphical (default)	python verify_dkim_bulk.py	Opens the configuration window
Command line	python verify_dkim_bulk.py --no-gui --src <folder> --out <folder> --mode <1|2|3>	Runs immediately (see Section 7)
Version	python verify_dkim_bulk.py --version	Prints the version and build date
Help	python verify_dkim_bulk.py -h	Lists command-line options

3. The Graphical Interface

The window is organised top-to-bottom in the order you configure a run.

3.1 Directory Workspace Configurations

Field	Default	Description
Source Folder	Current working directory	Folder containing the .eml files to verify. Use Browse... to pick a folder.
Output Folder	<current dir>/forensic_analysis_out	Where the CSVs, logs and artifacts are written. Created if it does not exist.

Documentation licensed under CC BY 4.0

3.2 Directory Scan & Output Sub-Artifact Toggles

Option	Default	Effect
Scan Target Folder Recursively	On	Includes .eml files in all sub-folders. When off, only the top level of the Source Folder is scanned.
Extract & Save Canonicalized Body Artifacts (bodies/)	On	Saves the exact canonicalised body bytes that were hashed, one file per signature.
Extract & Save Cryptographic Header Blocks (signatures/)	On	Saves the reconstructed signed header block (the data the signature covers, with b= emptied), one file per signature.
Strict Expiry: Treat Expired Signatures (x= tag) as FAIL	Off	When on, a signature whose x= expiry has passed is reported FAIL (Expired), as RFC 6376 requires of live mail receivers. When off (forensic default) it can still PASS, and the expiry is reported.

3.3 Public Key Mapping Strategy

Select one of three modes that determine where the public key comes from (Section 4).

Mode	Label in GUI	Key source
1	Automated Live DNS Resolution Query (w/ Local Cascading Fallbacks)	Live DNS, then local key files
2	Local 1:1 Sidecar File Matching Pattern	<EmailName>.key next to each .eml
3	Local N:1 Mapping Model	selector._domainkey.domain.key, then optional global key

Configure Global Manual Key Fallback (Optional Mode 3) opens a window where you paste key material (any format listed in Section 1) and click *Register Key Context*. This key is only used in Mode 3.

3.4 Content Threat Keyword Alerts Configuration

Ticking **Enable Advanced Body Content Keyword Flagging Alerts** reveals an editable box pre-filled with the default patterns (Section 6) and a **Load Custom Keywords File (.txt)** button that appends patterns from a text file. One regular expression per line.

3.5 Launch

LAUNCH TRIAGE PROCESSING LOOP validates the settings, closes the window and starts processing. Progress is shown in the console window. Before launching, the tool:

- Compiles every keyword line; a broken expression stops the launch and reports the offending line number.
- In Mode 3, warns if no global key is registered and no `*_domainkey*.key` files are found in the top level of the Source Folder, and asks whether to proceed.

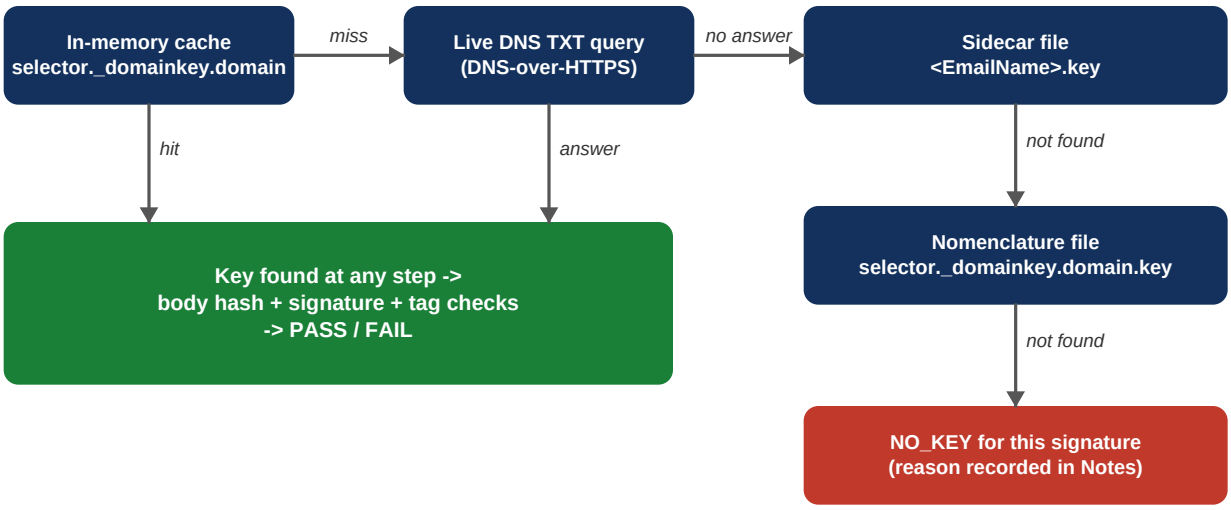
Documentation licensed under CC BY 4.0

4. Public Key Modes in Detail

DKIM keys are published in DNS at `<selector>._domainkey.<domain>`. Because domains rotate or retire keys, the key that signed an old email may no longer be in DNS. The three modes let you choose between the live record and keys you have preserved locally. When a message carries several signatures, the key is resolved separately for each signature's selector and domain.

Mode 1 - Live DNS with local fallbacks

Mode 1 - key resolution cascade (per DKIM signature)



- Lookups are sent over HTTPS to Google Public DNS (`https://dns.google/resolve`). Multi-string TXT records are joined, and if several key records are published the first is used and noted.
- Successful and failed lookups are cached for the run, so each selector/domain pair is queried only once.
- Lookup failures (network error, NXDOMAIN, no TXT record, no p= tag) are recorded in the Notes column and log before the local fallbacks are tried.
- Every key retrieved from DNS is saved to the output keys/ folder (Section 4.4), so the key as published at the time of the run is preserved even if the domain later rotates or removes it.

> **Privacy and reproducibility**
> Mode 1 sends the selector and domain of each message to a third-party resolver, and retrieves *today's* key, which may differ from the key in use when the email was sent.

Mode 2 - 1:1 sidecar files (offline)

Each email must have a key file with the same base name in the same folder. Only that file is used. If the email has several signatures, the sidecar key is tried against each; signatures made with a different key will report FAIL (the email still PASSES if one signature verifies).

```
evidence/
  Invoice_2026-03-14.eml
  Invoice_2026-03-14.key     <- key for this email only
```

Documentation licensed under CC BY 4.0

Mode 3 - N:1 nomenclature files (offline)

One key file per selector/domain, shared by every email signed with that key, stored in the same folder as the emails. If no matching file exists, the Global Manual Key (if registered) is used. This mode handles multi-signature messages best, because each signature finds its own key file.

```
evidence/
  msg001.eml   (d=example.com; s=selector1)
  msg002.eml   (d=example.com; s=selector1)
  selector1._domainkey.example.com.key   <- used for both
```

> **Tip: build a preserved key library**
> Capture the DKIM TXT record at intake (e.g. nslookup -type=TXT selector1._domainkey.example.com) and save it as a nomenclature .key file. Mode 3 then gives repeatable, offline results even after the domain rotates its key. A Mode 1 run does this for you: the keys/ folder it produces is already in nomenclature format.

Key_Source_Used values

Value	Meaning
CACHE_MEMORY	Mode 1 - key reused from an earlier lookup in this run
LIVE_DNS_QUERY	Mode 1 - key obtained from DNS
FALLBACK_SIDECAR_FILE	Mode 1 - DNS failed; <EmailName>.key used
FALLBACK_NOMENCLATURE_FILE	Mode 1 - DNS failed; selector._domainkey.domain.key used
SIDECAR_FILE	Mode 2 - <EmailName>.key used
N1_NOMENCLATURE_FILE	Mode 3 - selector._domainkey.domain.key used
GLOBAL_USER_SUPPLIED_KEY	Mode 3 - key pasted in the Global Manual Key window
UNKNOWN	No key located for the signature, or no signature present

Documentation licensed under CC BY 4.0

4.4 Preserving the keys used

In every mode, each distinct public key used during a run is saved to `<output>/keys/` as `<selector>._domainkey.<domain>.key`, containing the key record exactly as it was obtained. Every signature on every message is covered, so an email with three signatures contributes up to three keys. Each key is saved once per run, however many emails use it.

Item	Detail
keys/*.key	One file per distinct key. If the same selector/domain resolves to different key material during a run (e.g. DNS differs from a local file), both are kept and the second is named `...domain__<first 12 hex of SHA-256>.key`.
keys_manifest.csv	One row per saved key: Saved_Key_File, DKIM_Selector, DKIM_Domain, DNS_Name, Key_Source_Used, Original_Key_File_Path, First_Retrieved_UTC, Key_SHA256, Key_Type, Times_Used, Used_By (email#sigN list).
Saved_Key_File column	In both CSV ledgers; links each signature to its preserved key.
Log	Each signature section shows 'Key Preserved As'.

Re-running offline from preserved keys
Copy the files from keys/ into the folder holding the emails and run Mode 3. The results will match the original run regardless of later DNS changes, and Key_SHA256 will be identical. Revoked keys (empty p=) are saved too, as evidence of the state of DNS at the time.

5. Messages with Multiple DKIM Signatures

A message can carry several `DKIM-Signature` headers, and this is common. The tool examines **every** one of them; none are skipped.

Why messages have several signatures

Signer	Typical d=	Why
Author's own domain	example.org	The organisation signs its outgoing mail
Email service provider	mailer.example.net, sendgrid.net	Bulk/marketing platforms sign with their own domain as well
Mailing list or gateway	lists.example.com	Re-signs after modifying the message (Subject tag, footer)
Dual algorithms	example.org (twice)	Same domain signs with both RSA and Ed25519 keys

Each new signer adds its signature **above** the existing headers, so the top signature is the most recent and the bottom one is usually the first applied. The tool numbers them in file order: `Sig_Index 1` is the top signature.

How each signature is verified

Every signature is treated as a completely independent check, as RFC 6376 requires:

- its own public key, looked up from its own s= and d= (Section 4);
- its own body canonicalisation and l= limit, compared with its own bh=;

Documentation licensed under CC BY 4.0

- its own h= header list, rebuilt and checked against its own b= with its own algorithm (RSA or Ed25519);
- its own i=, t= and x= tags.

A failed signature does not cancel a valid one. Under RFC 6376 a failing signature is treated as if it were not there, so the email's `Overall_Status` is **PASS if at least one signature passes**.

What is recorded

Output	Multi-signature content
dkim_signature_details.csv	One row per signature with all fields and its own Signature_Status
Main CSV: Signature_Count / Signatures_Passed	e.g. 3 and 1
Main CSV: Reported_Signature	Which signature the per-signature columns show: the first that passed, otherwise the first that could be evaluated (e.g. '1 of 3')
Main CSV: All_Signatures	e.g. 1: d=lists.example.com ... -> PASS | 2: d=mailer.example.net ... -> FAIL | 3: ...
Log file	One section per signature, including the key record used
bodies/, signatures/	One artifact per signature, suffixed _sig1, _sig2, ...
keys/	The key used by each signature (Section 4.4)

Interpreting mixed results

Look at **which domain** passed, not just whether the email passed. A pass only for an intermediary's domain says less about the claimed sender than a pass for the domain in the From: header.

Pattern	Likely meaning
All signatures pass	Nothing covered by any signature has changed since the first signer
Only the top (list/gateway) signature passes	The intermediary changed the message (e.g. Subject tag, footer) and then re-signed. The content is vouched for by the intermediary only, from that point on
Author domain passes, intermediary fails	Usually an intermediary issue (key, configuration); the author's content is intact
None pass	Modified after the last signature, wrong/rotated keys, or not a faithful copy

Key modes and multiple signatures

Mode	Behaviour
1	Each signature's key is looked up in DNS; failures fall back to local files per signature
2	The one sidecar key is tried against every signature; signatures made with a different key report FAIL. Not recommended for multi-signature messages
3	Each signature finds its own selector._domainkey.domain.key file. Recommended

Documentation licensed under CC BY 4.0

Sample test pack

DKIM_MultiSig_TestPack contains five messages, each signed by three parties (bottom to top: example.org RSA, mailer.example.net RSA with a 7-day x= expiry, lists.example.com Ed25519), and the three key files. Run it in Mode 3 (or Mode 1, which falls back to the files):

File	Expected Overall_Status	Passed	Demonstrates
01_multi_dkim_all_pass.eml	PASS	3 of 3	Three signers, RSA and Ed25519
02_multi_dkim_list_modified.eml	PASS	1 of 3	List tagged Subject and added footer, then re-signed
03_multi_dkim_body_tampered.eml	FAIL	0 of 3	Amount changed in the body
04_multi_dkim_LF_line_endings.eml	PASS	3 of 3	LF-only file normalised to CRLF
05_multi_dkim_from_header_altered.eml	FAIL	0 of 3	From: domain altered (example.org to examp1e.org)

Signature 2 shows EXPIRED once its x= date (2026-11-12) has passed; with Strict Expiry on, that signature then reports FAIL (Expired), while the others are unaffected.

6. Keyword Alerts

Keyword alerts flag messages whose body matches investigator-defined regular expressions. They do not affect the DKIM result. Matching patterns are listed in the `Threat_Alerts` CSV column (or `CLEAN` if none match).

Default patterns

Theme	Pattern (case-insensitive)
Payment / banking	wire transfer | swift | routing number | bank account | invoice
Cryptocurrency	bitcoin | crypto | usdt | wallet phrase | private key
Credential phishing	password reset | verify your account | login expired | microsoft team
Legal / sensitive	subpoena | warrant | investigation | court notice | classified

Writing custom patterns

- One Python regular expression per line; blank lines are ignored.
- Prefix with `(?i)` for case-insensitive matching.
- Custom files are appended to whatever is already in the box - delete the defaults first if you do not want them.

```
(?i)(urgent payment|change of bank details)
(?i)\b(iban|bic)\b
(?i)gift ?cards?
```

Documentation licensed under CC BY 4.0

> **Important: the scan runs on the raw body**
> Patterns are matched against the undecoded MIME body. Text inside base64 or quoted-printable encoded parts and attachments will usually not match. Treat CLEAN as 'no match in raw text', not as proof the content is benign.

7. Command-Line Reference

Argument	Required with --no-gui	Description
--no-gui	-	Skip the window and run from the command line
--src <folder>	Yes	Source folder containing .eml files
--out <folder>	Yes	Output folder for the CSVs, logs and artifacts
--mode {1,2,3}	Yes	Public key mode (Section 4)
--strict-expiry	No	Treat signatures whose x= expiry has passed as FAIL (Expired)
--version	-	Print the version and build date
-h, --help	-	Show help

Fixed settings in command-line mode

Setting	CLI behaviour
Recursive scan	Always on
Save bodies/ and signatures/	Always on
Keyword alerts	Off (Threat_Alerts = CLEAN)
Global manual key (Mode 3)	Not available - provide nomenclature .key files instead

Examples

```
# Live DNS with local fallbacks
python verify_dkim_bulk.py --no-gui --src C:\Case123\eml --out C:\Case123\dkim --mode 1

# Fully offline, one shared key file per selector/domain, strict expiry
python verify_dkim_bulk.py --no-gui --src ./eml --out ./dkim_out --mode 3 --strict-expiry
```

Documentation licensed under CC BY 4.0

8. How Verification Works

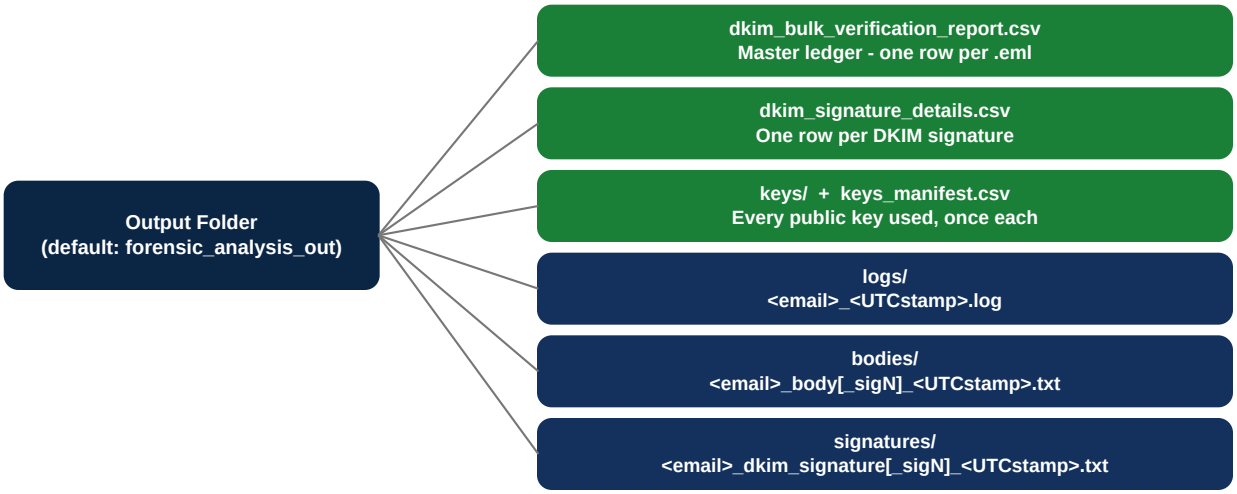
Step	Operation	Detail
1	Hash the file	MD5 and SHA-256 of the unmodified .eml bytes
2	Normalise line endings	Bare LF converted to CRLF on a working copy (reported in Line_Endings)
3	Split headers / body	At the first blank line
4	Find signatures	Every DKIM-Signature header, in order (top = most recently added)
5	Evaluate tags	i= must be d= or a subdomain of it; t= and x= converted to UTC; x= must be later than t=
6	Get public key	According to the selected mode, per signature
7	Canonicalise body	simple or relaxed per c=; truncated to l= if present
8	Body hash	SHA-256 (SHA-1 for rsa-sha1), base64; compared with bh=
9	Rebuild signed block	Headers in h= order (bottom-up for repeated names), then the DKIM-Signature with b= emptied
10	Verify signature	RSA: PKCS#1 v1.5 with SHA-256/SHA-1 over the block. Ed25519: over the SHA-256 digest of the block (RFC 8463)
11	Decide	Per signature PASS / FAIL; email PASSES if any signature passes
12	Record	Both CSVs, log file, optional bodies/ and signatures/ artifacts

Line-ending normalisation
SMTP transmits mail with CRLF line endings, and DKIM is calculated over CRLF data. Some export tools save .eml files with LF only. The tool converts bare LF to CRLF in memory before verifying. Hashes, the original file and Line_Endings reporting are unaffected; the saved bodies/ and signatures/ artifacts reflect the normalised data.

Documentation licensed under CC BY 4.0

9. Output and Interpreting Results

Output folder layout



- Timestamps are UTC in the form YYYYMMDDTHHMMSSZ, so repeated runs do not overwrite logs or artifacts.
- Artifact names include `_sigN` when an email has more than one signature.
- All CSVs and the keys/ folder contents are overwritten on each run to the same Output Folder - use a new folder per run to keep history.
- A log is written for every signed email, including those with missing keys; it lists every signature, the key record used, the t= time and its offset from the Date header.

Main report columns (dkim_bulk_verification_report.csv)

Where an email has several signatures, the signature-specific columns describe the **reported signature**: the first one that passed or, if none passed, the first one that could be evaluated.

Column	Content
EML_Filename	File name of the message
MD5_Hash / SHA256_Hash	Hashes of the original .eml file
Line_Endings	CRLF, LF (normalised to CRLF) or MIXED (normalised to CRLF)
Signature_Count / Signatures_Passed	Number of DKIM signatures found and how many passed
Reported_Signature	Which signature the following columns describe, e.g. '2 of 2'
DKIM_Domain / DKIM_Selector / DKIM_Algorithm	d=, s= and a= tags
DKIM_Identity / Identity_Check	i= tag (or the @d= default) and whether it is within d=
Sig_Timestamp_UTC / Sig_Expiry_UTC	t= and x= converted to UTC
Expiry_Status	NO EXPIRY, VALID (not expired), EXPIRED (at verification time) or INVALID (...)
Key_Source_Used / Key_File_Path / Key_SHA256	Where the key came from, local file used, SHA-256 of the key text

Documentation licensed under CC BY 4.0

Column	Content
Saved_Key_File	Preserved copy of the key in keys/ (Section 4.4)
Body_Hash_Result	MATCH / MISMATCH / ERROR / N/A
Signature_Result	VERIFIED / FAILED / ERROR / N/A
Overall_Status	See status table below
All_Signatures	One-line summary of every signature and its status
Threat_Alerts	Matching keyword patterns, or CLEAN
Notes	DNS lookup results, expiry notes, l= coverage, deprecated algorithm warnings

Signature details (dkim_signature_details.csv)

One row per signature with the same fields, plus `Sig_Index`, `Canonicalization` and `Signature_Status`. Use it when an email has several signatures or to filter by domain (Section 5).

Status values

Overall_Status (per email)	Meaning
PASS	At least one signature passed every check
FAIL	No signature passed (hash/signature failure, invalid i= or x= tag)
FAIL (Expired)	Strict Expiry on; signatures would otherwise pass but have expired
UNVERIFIED (No Signature)	Message has no DKIM-Signature header
UNVERIFIED (Missing Public Key)	No key found for any signature
RUNTIME_ERROR (...)	Processing error, e.g. malformed file, revoked/invalid key; message included

Signature_Status (per signature)	Meaning
PASS	Body hash MATCH, signature VERIFIED, i= valid, x= valid (and not expired in strict mode)
FAIL	Body hash mismatch and/or signature failed
FAIL (Invalid i= tag)	Cryptographically valid, but i= is outside the d= domain (RFC 6376 s3.5)
FAIL (Invalid x= tag)	Cryptographically valid, but x= is not later than t= (or not a number)
FAIL (Expired)	Strict Expiry only: valid but x= has passed
NO_KEY	No public key found for this signature
ERROR (...)	The signature could not be evaluated; reason included

Documentation licensed under CC BY 4.0

Reading a result

Body hash	Signature	Likely interpretation
MATCH	VERIFIED	Signed headers and body unchanged since signing
MISMATCH	VERIFIED	Signed headers intact but body changed - e.g. edited content, or altered in transit by a gateway/disclaimer/footer
MATCH	FAILED	Body intact but a signed header changed, wrong/rotated key, or export altered headers
MISMATCH	FAILED	Message modified, wrong key, or file not a faithful copy

Expired is not the same as altered
Many senders set x= only days after t=, so older evidence is often expired at verification time. An expired but cryptographically valid signature still shows the message has not been changed. Keep Strict Expiry off for forensic work, and use the log's 't= minus Date' value to compare the signing time with the Date header.

A FAIL is not automatically proof of tampering
Legitimate causes include mailing-list or security-gateway modification, key rotation (Mode 1 retrieves today's key) and exports that re-encode the message. Corroborate with headers, server logs and other copies of the message.

Documentation licensed under CC BY 4.0

10. Known Limitations (v1.7.1)

Area	Observation	Impact / workaround
Mode 2 with several signatures	A single sidecar key is tried against every signature.	Signatures from other domains report FAIL; use Mode 3 for multi-signature messages.
Keyword scan	Runs on the raw, undecoded body.	Encoded content will not match (Section 6).
Mode 3 pre-check	The GUI warning looks only at the top level of the Source Folder.	Nomenclature files must sit beside the emails they apply to.
Policy checks	DMARC/SPF, From-domain alignment and whether h= includes From are not assessed.	Compare DKIM_Domain with the From header manually when impersonation is in question.
Mode 1	Retrieves the current key from a third-party resolver.	Prefer Mode 3 with keys preserved at intake for repeatable results.

Good practice

- Work on copies of evidence; record the SHA-256 values in the CSV against your intake hashes.
- Prefer Mode 3 with key records preserved at intake for repeatable results.
- Keep the CSVs, logs and artifacts from each run together in a dedicated output folder.

11. Version History

Version	Date	Changes
1.7.1	2026-11-05	Every public key used is preserved in keys/ with keys_manifest.csv; new Saved_Key_File column; 'Key Preserved As' in logs. Caches reset per run. Guide: new Section 5 on messages with multiple DKIM signatures and the multi-signature test pack.
1.7.0	2026-11-05	Fixed Mode 1 DNS-over-HTTPS URL; DNS errors reported in Notes; failed lookups cached; multi-string TXT records joined. All DKIM-Signature headers verified; new dkim_signature_details.csv and Signature_Count, Signatures_Passed, Reported_Signature, All_Signatures columns. LF-only and mixed line endings normalised to CRLF; new Line_Endings column. Ed25519 signatures verified over the SHA-256 digest of the header block (RFC 8463). t=, x= and i= tags evaluated; new identity and expiry columns; Strict Expiry option (GUI and --strict-expiry). Logs list every signature, the key record used and t= offset from the Date header. Added Notes column, --version option and Apache 2.0 header.
1.6.2	2026-11-05	Initial documented release

Documentation licensed under CC BY 4.0

12. License, Warranty and Repository

Repository

The script, this guide and the README are published at https://github.com/jjrboucher/DKIM-Validator. Check the repository for the latest version, release notes and issue reporting.

Licensing

Component	License	Summary
Software (verify_dkim_bulk.py)	Apache License 2.0	Free to use, modify and distribute, including commercially. Keep the copyright and license notices, state significant changes, and include a copy of the license and the NOTICE file. Includes an express patent grant.
Documentation (this guide, README)	Creative Commons Attribution 4.0 International (CC BY 4.0)	Free to share and adapt for any purpose, provided appropriate credit is given to the author, a link to the license is included and any changes are indicated.

Full license texts: apache.org/licenses/LICENSE-2.0 and creativecommons.org/licenses/by/4.0.

> **Disclaimer of warranty**
> This tool is provided free of charge, "AS IS", WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including any warranties of merchantability, fitness for a particular purpose, accuracy or non-infringement. In no event shall the author be liable for any claim, damages or other liability arising from the use of the tool or its results. Users are responsible for validating results and for the evidential weight they place on them.

Suggested attribution for this guide

DKIM Bulk Signature Verifier - User Guide (v1.7.1) by Jacques Boucher, licensed under CC BY 4.0. https://github.com/jjrboucher/DKIM-Validator

Tool	DKIM Bulk Signature Verifier
Version	1.7.1
Build date	2026-11-05
Script	verify_dkim_bulk.py
Author	Jacques Boucher (jjrboucher@gmail.com)
Repository	https://github.com/jjrboucher/DKIM-Validator
Licenses	Software: Apache-2.0 | Documentation: CC BY 4.0